

Machine Learning – HW#3

General notes:

- It is **recommended** that you work alone to learn the material. However, groups of size 2 are allowed.
- Read and understand the questions, and answer accordingly. Be sure to answer what is asked. When in doubt, you can provide more details.
- Some of the questions require material not covered in class, or will be covered in future lectures. This is intentional.
- **Warning:** do not copy other student work. This includes code similarity. Do not sit and solve assignments together, unless it is your group partner.

Submission document format guidelines:

- Submit pdf document to Gradescope.
- You can use word, latex, or any other editor.
- No need for cover page. Provide your names and IDs in the header.
- Answer each Question on a new page. No need to copy the question, just put Question # at the beginning, then a), b), etc. for each sub-question.
- When submitting to Gradescope, you will be given the chance to map pages to questions.
- For groups, one student submits and adds their teammate during submission on Gradescope.

Question 1 [20 pts]

You are given Boston housing price **regression** dataset in 'Boston.csv'. The last column `medv` is the prediction label, and the other columns are **continuous** features.

NOTE: for each of the following parts show the *relevant* code *snippet* and print results.

a) Do the following in the list order:

- Read in the dataset and construct X feature matrix and y label vector.
- Use sklearn's [train_test_split](#) to split (X, y) into (X_train, y_train) and (X_test, y_test) where train-test ratio is 80%-20% (i.e. `test_size=0.2`). Also, use `random_state=11`.
- Further split (X_train, y_train) into (X_train, y_train) and (X_val, y_val) where `test_size=0.2` and `random_state=11`.
- Print the shapes to confirm you got this part correctly. The below is my implementation's shapes.

```
Shape (X, y): ((506, 12), (506,))
Shape (X_train, y_train): ((323, 12), (323,))
Shape (X_val, y_val): ((81, 12), (81,))
Shape (X_test, y_test): ((102, 12), (102,))
```

b) We will use the validation set for model selection from among the following 4 options:

1. [Standard scale](#) the features and default linear regression.
2. [Polynomial featurization](#) of degree 2, then standard scale the features, and default linear regression.
3. Polynomial featurization of degree 3, then standard scale the features, and default linear regression.
4. Polynomial featurization of degree 4, then standard scale the features, and default linear regression.

I recommend you use sklearn [pipeline](#) for ease-of-use. For example, (2) can be done as follows:

```
pipeline_2= make_pipeline(PolynomialFeatures(degree=degree, include_bias=False),
                           StandardScaler(),
                           LinearRegression())
```

Note: you should set `include_bias=False`, since `LinearRegression` adds bias by default.

Train the 4 options on the training set, then compute [mean-absolute-error](#) (MAE) on the validation set. Print the results and clearly state which is the best option. Below are my results:

```
Validation MAE results:  
degree_1: 3.1541726681084916  
degree_2: 2.8070726992533346  
degree_3: 13.880448190233244  
degree_4: 8.021046452607147
```

c) After determining the best option from part (b), do the following:

- Merge the training set and validation sets into one. Numpy's [vstack](#) and [hstack](#) are helpful for this.
- *Re-train* the best option on the merged dataset.
- Compute MAE performance on the test set.

My results:

```
Final model PolyLinearReg degree=2, test mae 2.883085542547702
```

Question 2 [20 pts]

You are given the stroke **classification** dataset in 'stroke.csv'.

The **continuous** features are: age, avg_glucose_level, and bmi. The rest are **discrete** features. The **label** stroke is binary {0, 1}.

NOTE: for each of the following parts show the *relevant* code *snippet* and print results.

a) Do the following in the list order:

- Read in the dataset. **We will treat the entire dataset as training.**
- Take the discrete features and [one-hot](#) encode them.
- Take the continue features and [standard scale](#) them.
- Construct X_train by hstacking the two feature matrices above. Also, setup y_train.
- Print the shapes to confirm you got this part correctly. The below is my implementation's shapes.

```
(X_train, y_train) shape: ((5109, 23), (5109,))
```

b) Code the function

```
def logreg_gradient_descent(X, y, learning_rate=0.01, max_iters=1000)
```

that implements logistic regression full-batch gradient descent. The function returns two items: the fitted weight vector (w_hat) and list of iteration log losses (loglosses). Do it as follows:

- First set np.random.seed(11), then initialize w_hat as a sample from a normal distribution: np.random.normal(loc=0.0, scale=1, size=X.shape[1])
- Setup an empty list for loglosses
- Loop for max_iters to do:
 - Compute prediction probabilities on X using current w_hat.
 - With predictions, you can compute current iteration [log_loss](#)(y, y_preds). Append this iteration log loss to loglosses list.
 - Update w_hat using logistic regression gradient update.
- After loop return w_hat, loglosses

Just show snippet for your function implementation.

c) Now let's use the function you implemented in part (b). Use the two following statements:

```
w_hat_1, loglosses_1 = logreg_gradient_descent(X_train, y_train, learning_rate=0.01, max_iters=500)
w_hat_2, loglosses_2 = logreg_gradient_descent(X_train, y_train, learning_rate=0.001, max_iters=500)
```

Make note of the arguments. After running these two statements, use matplotlib or seaborn to [plot](#) the learning curves of iteration loglosses_1 and loglosses_2. Make two separate plot figures.

Show your code snippet for this part, and the two figures. What is the issue with the first figure, and briefly explain **why** this issue occurred?

d) Now let us inspect label predictions. Using w_hat_2 from part (c), do the following:

- Predict the probabilities on X_train
- Force a label prediction with threshold of 0.5. Then use [ConfusionMatrixDisplay.from_predictions](#) to plot the confusion matrix.
- Now try another label threshold of 0.1. Then plot the confusion matrix.

Show code snippet for this part, and show the two confusion matrix plots.

Give a brief explanation **why** a threshold of 0.5 is giving bad TP performance, and **how** we can fix this?

For reference here are my confusion matrix plots.

